University of East London

Log anomaly detection on BGL
Five project questions (detailed answers)

Student name: Subhan Jameel

Student ID: U2926092

MSc Artificial Intelligence

CN7000 - MSc Project Dissertation

Repository: ml_pipeline + log_anomaly_rails

The following sections answer five questions about the dissertation project, based on the implemented codebase (training pipeline, Flask API, Rails dashboard).

BGL Log Anomaly Detection Project

Five detailed answers (repository: ml_pipeline + log_anomaly_rails)
Based on implementation: trainer.py, data_loader.py, bert_log_model.py, api/app.py, Docker, Rails MlApiService

1. Why I chose this project

Real operational need

Large-scale systems produce huge continuous log streams. Early failure signals often appear as subtle patterns across many lines before outages become visible in simple metric dashboards. Traditional rule-based alerting misses novel or composite faults. This project addresses that gap with machine learning that learns from data rather than only from hand-written rules.

Strong research and benchmarking foundation

The work is anchored on the Blue Gene/L (BGL) supercomputer log corpus from LogHub, a widely cited public benchmark. That makes results comparable to published papers on log parsing (e.g. Drain), classical ML, deep sequence models, and transformer-based detectors. The repository includes MODERN_MODELS_LITERATURE.md and model docstrings that tie code choices to peer-reviewed sources (BERT-Log, NeuralLog, LogBERT, LogFormer, LogGPT, etc.).

Fits an MSc AI dissertation structure

The project combines supervised learning, imbalanced classification, NLP-style transformers, explainability (SHAP for tabular models), honest evaluation (chronological train/validation/test split in AnomalyDetectionTrainer._chronological_split), and a complete software deliverable: a Python training and inference service plus a Rails dashboard. It demonstrates end-to-end ML engineering, not only notebook experiments.

Clear separation of concerns for assessment

Training lives in ml_pipeline; the web tier in log_anomaly_rails talks to Python only through MlApiService and HTTP. That mirrors how industry systems are organised and is easy to explain in a viva: data path, model path, API contract, and UI.

Ethical and data practicality

BGL is anonymised research data (LogHub). The codebase can also generate a BGL-style proxy dataset when BGL.log is not present, so development and demos remain possible without redistributing multi-gigabyte files in every environment.

Skill coverage for an AI master's portfolio

The project exercises the full stack employers expect from "ML engineer" roles: Python scientific stack (NumPy, scikit-learn, PyTorch/Hugging Face transformers where installed), data cleaning and labelling logic, experiment tracking via JSON artefacts, containerisation (Dockerfile + compose), REST API design, a second language stack (Ruby on Rails) for product-facing code, and written technical documentation under

log_anomaly_rails/docs and ml_pipeline in-line module docs.

2. What is the future of this project

Scale training to the full BGL corpus on GPU

trainer.py can load real BGL from ml_pipeline/data/BGL.log (or via scripts/download_bgl.py). Current training may use a sampled subset (e.g. up to 400k lines in run_full_pipeline) for tractability on CPU. A natural extension is full-corpus training with GPU and longer fine-tuning so results align more closely with published full-data F1 while keeping chronological evaluation.

Stronger evaluation and monitoring

log_anomaly_rails/docs/metrics-interpretation.md explains that small holdouts can show deceptively perfect accuracy. Future work should enlarge the held-out test set, add k-fold or repeated runs where feasible, and report confusion counts alongside headline metrics. Operational monitoring could add Prometheus-style latency and error metrics from Flask and Rails.

Production hardening

SECURITY.md and deployment-notes in the repo imply the coursework stack assumes a trusted lab network. Future iterations would add authentication, TLS, mutual trust between Rails and the ML API, secrets management, and immutable audit logs for scored windows (important for SOC workflows).

Streaming and near-real-time ingestion

The current design is oriented to windowed batch scoring over log lines and REST predict endpoints. A production evolution would attach to a log shipper (Kafka, Fluent Bit), maintain rolling windows in memory or a stream processor, and tune latency under load.

Multi-dataset generalisation

data_loader.py and documentation reference BGL specifically. Extending the same pipeline to other LogHub corpora (e.g. HDFS) with one configuration would test cross-system robustness - a known open problem when parsers and vocabularies drift.

Model registry and shadow deployment

training_metadata.json already records best_model and per-model metrics. A registry could version artefacts, store data checksums, and run shadow scoring (new model vs champion) before switching PRIMARY_INFERENCE_MODEL in api/app.py from BERT-Log to another checkpoint.

3. Why we use this model (and this model family)

Primary inference model: BERT-Log (DistilBERT)

api/app.py sets PRIMARY_INFERENCE_MODEL = "BERT-Log". The active scorer for dashboards and predict is this transformer-based classifier. bert_log_model.py uses distilbert-base-uncased with a binary classification head, max length 256, fine-tuning hyperparameters suited to limited compute (e.g. batch size 16, learning rate 2e-5, a small number of epochs in code). Inputs are raw concatenated log text per window, so WordPiece tokenisation captures semantics without relying solely on discrete template IDs - the same design philosophy as BERT-Log and NeuralLog in the literature cited in the file header.

Why transformers here, not only Random Forest or Logistic Regression

Classical models in this repo consume an engineered matrix from FeatureEngineer: TF-IDF on Drain templates plus severity, component, and temporal statistics (83 dimensions combined). That path is fast and interpretable (SHAP in trainer.py for forest and logistic). But count and bag-of-template features lose ordering and subtle wording; similar-looking messages can differ in meaning. Transformers leverage pretrained language representations and context, which published benchmarks on BGL (see MODERN_MODELS_LITERATURE.md and comments in bert_log_model.py) associate with higher F1 than Drain+LR/RF alone.

Why keep multiple models at all

trainer.py trains nine candidates: four feature-matrix models (Logistic Regression, Random Forest, Isolation Forest, LSTM Autoencoder) and five text models (BERT-Log, LogBERT, PLELog, LogFormer, LogGPT). This supports fair comparison under one chronological split, illustrates precision/recall/latency trade-offs (e.g. conservative Random Forest vs balanced BERT-Log), and supports dissertation claims with evidence rather than a single black box.

Fallback policy

If the best model F1 falls below F1_THRESHOLD (0.88), the trainer can substitute an optimised Random Forest path so demos do not collapse when a heavy model fails on a given machine. That is an engineering safety net, not a claim that forests are always superior.

Calibration detail

BERTLogModel stores decision_threshold (default 0.5, updated from validation during training when implemented) so predicted labels from predict() align with calibrated probabilities from predict_proba where available; AUC-ROC in evaluator.py still ranks anomalies using continuous scores - this split matches honest thesis reporting (no threshold tuning on the test fold).

4. What dataset we train this model on

Primary dataset: BGL (Blue Gene/L)

data_loader.py loads real BGL logs from a file such as ml_pipeline/data/BGL.log. Lines follow the BGL layout parsed by DrainParser (label, timestamp, node, RAS/APP types, component, message). Labels drive is_anomaly from the leading token (- for normal in typical BGL convention vs alert types). The loader can sample lines for faster iteration (e.g. load_sample with a cap) as implemented in the training pipeline.

Where the data comes from

The project expects LogHub-style distribution; scripts/download_bgl.py can fetch data when configured. The dissertation context uses millions of lines and hundreds of thousands of alert-labelled rows when the full file is used.

Unit of training: sliding windows

AnomalyDetectionTrainer uses WINDOW_SIZE = 20 and STRIDE = 10. Each training example is a window of consecutive log lines; the window label reflects anomaly presence inside that slice. The same windows are turned into (a) numeric features for classical models and (b) concatenated text strings for transformer models, so comparisons are aligned.

Train / validation / test protocol

TEST_SIZE = 0.20 and VAL_SIZE = 0.15. The split is chronological: the last portion of windows in time order is held out for testing to reduce leakage that random splitting can introduce when adjacent windows repeat patterns from the same incident (as discussed in replication literature; the codebase comments reference this explicitly).

Class imbalance

Anomalies are typically a small fraction of lines. SMOTE is applied on the feature-matrix training data for supervised classical models. BERT-Log uses weighted loss appropriate to imbalance (documented in bert_log_model.py).

Proxy dataset when BGL.log is missing

If no real file is found, BGLDataLoader.generate_bgl_proxy builds synthetic BGL-like traces so the pipeline and API still run for development. Metrics and thesis claims should be based on real BGL when reporting "production-like" performance; proxy mode is for smoke tests and missing-data fallback, logged clearly in training metadata.

5. What is the outcome of this project

Research and comparative outcomes

The pipeline trains and evaluates nine models on the same chronological split and reports precision, recall, F1, accuracy, AUC-ROC where available, false positive/negative rates, and per-sample latency via ModelEvaluator in evaluator.py. Results are written to training_metadata.json and holdout bundles eval_holdout.npz and eval_holdout_text.npz for matrix and text models respectively. This yields an evidence-backed ranking (typically favouring BERT-Log on balanced F1 in the dissertation narrative) while preserving classical baselines for interpretability and conservative precision.

Software artefacts

ml_pipeline: training script (scripts/train_pipeline.py), saved pickles under saved_models/, Flask REST API (api/app.py) exposing health, models, metrics, predict, batch predict, explain, simulate. log_anomaly_rails: dashboard, analysis views, alerts, performance page, Monitor stream; integration via MlApiService with offline fallbacks when the API is down.

Deployment story

docker-compose.yml runs ml_api on port 5001 and rails_app on 3000 with ML_API_URL wiring. start.sh can start Flask then Rails on a developer machine for demonstrations without Docker.

Explainability

SHAP explainers are trained for Random Forest and Logistic Regression when training succeeds, with optional global importance exported. Transformer interpretation in the thesis may use separate visual methods; the API includes an explain route for integrated tooling.

Honest reporting

Internal docs (metrics-interpretation.md) caution that tiny holdouts can show accuracy 1.000 without implying perfection on the entire BGL file. The outcome includes not only numbers but documentation of limitations -

appropriate for MSc-level scientific integrity.

Educational outcome

The student delivers a reproducible repo tying together log parsing, feature engineering, imbalanced learning, deep and transformer models, REST integration, and a web UI - a capstone outcome for an MSc in Artificial Intelligence focused on applied machine learning systems.

Technical reference (code-level)

DrainParser (drain_parser.py): depth 4, similarity threshold 0.4 for BGL template mining; optional drain3 backend with matching config; BGL line regex extracts label, timestamp, node, RAS/APP fields, component, message.

FeatureEngineer (feature_engineering.py): up to 50 TF-IDF features on template-derived text per window plus statistical features (severity moments, component histogram across TOP_COMPONENTS, duration, event rate, template diversity, repeat patterns) scaled with StandardScaler - combined width 83 in code comments and thesis.

ModelEvaluator (evaluator.py): binary precision, recall, F1, accuracy, ROC AUC when predict_proba yields scores, confusion-matrix TN/FP/FN/TP, FPR/FNR, latency ms per sample; metrics match predict() paths used by live /api/v1/predict, not a separately tuned threshold on the test fold.

Flask API (api/app.py): GET /api/v1/health, /api/v1/models, /api/v1/metrics; POST /api/v1/predict, /api/v1/predict/batch, /api/v1/explain, /api/v1/simulate; holdout metrics refresh when eval_holdout.npz or related files change (signature from file mtimes and key source files).

Rails bridge (ml_api_service.rb): ML_API_URL default http://localhost:5001; 30s timeout; health_check, predict, get_metrics, get_models, explain, simulate; rescue paths return offline stub JSON so the UI remains usable if Python is down.

Nine trained artefacts (trainer.py train_all_models): Logistic Regression, Random Forest, Isolation Forest, LSTM Autoencoder, BERT-Log, LogBERT, PLELog, LogFormer, LogGPT - saved as .pkl (and companion files) under saved_models/ with training_metadata.json summarising runs.